



NEXUS

NEXUS TECH RECRUITMENT

2026

TECHNICAL DEPARTMENT · CLUB RECRUITMENTS

**BUILD SOMETHING
THAT MAKES STUDENT LIFE EASIER.**

A beginner-friendly, project-based recruitment challenge. AI and vibecoding are encouraged — understanding what you build matters more than how much code you write.

01 • PROBLEM STATEMENTS

Choose ONE problem statement from the section corresponding to your year of study. The descriptions below define the problem and expected direction of the solution. You may make sensible implementation choices while ensuring that the core requirements are addressed.

WHAT WE LOOK FOR

Problem understanding · logical thinking · practical implementation · thoughtful use of technology · ability to explain your own work

Problem statement list

#	ELIGIBILITY	PROBLEM STATEMENT
01	1st year	Smart Product Recommendation System
02	1st year	Smart Document Knowledge Assistant
03	2nd / 3rd year	Real-Time Air Quality Monitoring and Prediction for User-Selected Regions
04	2nd / 3rd year	Self-Optimizing Web Application Using Machine Learning

Year-wise selection

1st years: Select either Problem 01 or Problem 02.

2nd / 3rd years: Select either Problem 03 or Problem 04.

1ST YEAR • PROBLEM 01

Smart Product Recommendation System

An online shopping platform wants to develop a Machine Learning based product recommendation system. It has historical customer-product interaction data containing Customer ID, Product ID, Product Category, Product Price, number of times the product was viewed, number of times it was added to the cart, number of times it was purchased, and the customer's previous purchases.

The company wants the system to recommend products that a customer is likely to purchase.

Design and implement a complete working Machine Learning solution for this problem. The solution should process and clean the historical data, handle missing and duplicate records, perform suitable feature engineering, and train an appropriate Machine Learning model to predict whether a customer is likely to purchase a particular product.

The system should evaluate the trained model and then accept a customer ID or customer details as input. It should generate predictions for available products, calculate their predicted purchase probability, rank the products, and display the Top 5 recommended products.

The final implementation should be a working recommendation system that generates personalized recommendations for a new customer.

1ST YEAR • PROBLEM 02

Smart Document Knowledge Assistant

Context & Problem

Students and researchers frequently handle large collections of unstructured files—such as lecture PDFs, research articles, lab manuals, and text notes. Finding specific answers across multiple documents using traditional keyword search (Ctrl+F) is slow and ineffective because it cannot understand concept context, synonyms, or multi-page relationships.

Objective

Design and build a web-based Knowledge Assistant for first-year students that ingests uploaded documents (.pdf, .txt) and uses Retrieval-Augmented Generation (RAG) to provide fast, accurate, and context-aware answers grounded strictly in the provided files.

Core Requirements

1. File Upload & Processing

Build a user interface allowing users to upload and manage single or multiple document files (.pdf, .txt).

Also receive a user query, perform a similarity search to retrieve the top 3–5 most relevant text chunks from it.

2. Context-Aware AI Generator

Pass the retrieved text snippets as context alongside the user's query to a Large Language Model API.

3. Interactive User Interface

Display a chat interface where users can ask follow-up questions.

Show referenced document snippets alongside the AI answer so users can verify the source text.

Expected Output

A functional Python web app.

A live demonstration uploading sample lecture notes and answering specific questions based on the uploaded content.

2ND / 3RD YEAR • PROBLEM 03

Real-Time Air Quality Monitoring and Prediction for User-Selected Regions

Air pollution levels vary significantly across different geographical regions and change over time due to factors such as traffic, weather, industrial activity, and seasonal conditions. However, users often have difficulty understanding the current and expected air-quality conditions of a specific area and whether the region has previously experienced documented air-quality or environmental compliance issues.

The proposed system will provide an interactive platform where users can select a geographical region and specify a time period to view available air-quality information from open data sources. The system will analyze the collected data to identify pollution trends and use a machine-learning model to predict air-quality levels for a future time period.

In addition, the system will retrieve publicly available information about environmental or air-quality issues associated with the selected region and present relevant best practices for reducing exposure to pollution and improving local air-quality conditions.

Main Objectives

- Allow users to select a location or region on a map.
- Retrieve real-time/open-source air-quality data for that region.
- Allow users to specify a time window for analysis.
- Analyze historical/current pollution trends.
- Predict future air-quality levels using a basic ML model.
- Display pollutants such as PM_{2.5}, PM₁₀, NO₂, SO₂ and O₃, where available.
- Retrieve publicly available environmental/compliance-related information associated with the selected area.
- Present practical best practices and recommendations based on the observed pollution conditions.

2ND / 3RD YEAR • PROBLEM 04

Self-Optimizing Web Application Using Machine Learning

Problem

Most web applications are reactive: when traffic increases, database queries slow down, or response times rise, the application may become sluggish before developers identify and resolve the issue. This can result in poor user experience and reduced system performance.

Solution

Build a full-stack web application with an embedded ML-based monitoring engine that analyzes application performance, predicts potential bottlenecks, and automatically applies optimization strategies before users experience significant slowdowns.

The ML model is trained using an existing web/application performance dataset containing parameters such as traffic volume, response time, CPU utilization, memory utilization, database query time, active users, and system load.

How It Works

Observe: Tracks real-time or simulated application metrics such as traffic, request response times, CPU/memory utilization, active users, and database query speeds through a monitoring dashboard.

Predict: The trained ML model analyzes incoming performance metrics and predicts upcoming high-load conditions or performance degradation.

Auto-Optimize: Based on the ML prediction, the system automatically applies predefined optimization strategies such as enabling caching, reducing database pagination limits, optimizing database queries, or disabling heavy non-essential page components during high-load periods.

Recover: Once the system detects that the load has returned to normal, it can automatically restore the application's standard configuration.

Novelty

A self-optimizing web application that uses an ML model trained on historical performance data to anticipate load issues and automatically reconfigure application-level behavior before significant performance degradation occurs.

The system follows a continuous:

MONITOR



PREDICT



OPTIMIZE



RECOVER

cycle, creating a practical demonstration of intelligent and self-healing web application performance management.